

DSL504: Natural Language Processing (NLP)

Group Assignment 1

Group Information

- Group Number: 10
- Submission Type: Report + Implementation

Group Members(in alphabetical order)

#	Name	Roll Number
1	Aditya Jha	12340090
2	Aditya Yadav	12340100
3	Hirannya Mhaisbadwe	12340950
4	Niraj Chandekar	12341480
5	Sanjani Kumari	12341890

Question 1: Segmentation + POS Tagging + Beam Decoder

1. Objective

This question focused on building a reusable Q1 runtime module for token segmentation and POS tagging using the existing trained model instead of retraining.

2. What was implemented

- `q1_module.py` exposes:
 - `segment_beam(token: str) -> list[str]`
 - `pos_tag(words: list[str]) -> list[tuple[str, str]]`
 - `trigram_lm` as the loaded trained model object
- Offline training logic remains in `q1_train.py`.
- The model weights/artifacts are reused directly without retraining during inference.

3. Reuse requirement

- Q1 segmentation and POS tagging are reused in Q4 as part of the live correction pipeline.

- The Q1 trigram LM is reused as-is for downstream scoring and segmentation checks.
 - Training is not repeated in the final pipeline.
-

Question 2: Dependency Parser

1. Objective

This question required writing a dependency parser using a transition-based approach and training it on a suitable treebank.

2. Work completed

- Implemented a transition-based parser using the standard dependency parsing pipeline.
- Used a treebank-based training setup and oracle-based transition generation.
- Parsed sample sentences and evaluated dependency structure using standard parser metrics.

3. Reflection

This portion of the assignment contributed the syntactic structure perspective and helped frame the later grammar-checking and final scoring decisions.

Question 3: Spelling Corrector

1. Objective

This question focused on building a spelling corrector using vocabulary and language-model-based corrections without retraining at inference time.

2. What was implemented

- `q3_module.py` exposes:
 - `vocab`
 - `unigram_freq`
 - `bigram_model`
 - `gen_candidates_a(word)`
 - `gen_candidates_b(word)`

- `correct_nonword(word)`
- `correct_realword(sentence, target_idx)`
- The vocabulary and language-model statistics are loaded from previously trained artifacts and reused directly.

3. Reuse requirement

- Q3 outputs are reused in Q4 as the first-stage spelling correction layer.
 - Q4 does not retrain the vocabulary or n-gram model.
 - The correction layer is used both for unknown words and for local real-word correction checks.
-

Question 4: Integrated NLP Pipeline for Typing Errors, Grammar Alerts, and Final Analysis

1. Scope completed

This assignment was implemented as a single reusable pipeline across Q1, Q3, shared LM, live typing, PCFG parsing, and final sentence scoring:

- Q1 runtime refactor: `q1_module.py` exposes `segment_beam(token)`, `pos_tag(words)`, and the reusable `trigram_lm`, while training logic remains in `q1_train.py`.
- Q3 runtime refactor: `q3_module.py` exposes `vocab`, `unigram_freq`, `bigram_model`, `gen_candidates_a()`, `gen_candidates_b()`, `correct_nonword()`, and `correct_realword()`, with no retraining at inference time.
- Shared LM: `shared_lm.py` builds one Brown-based add-k model once and reuses it for live grammar checking and final scoring.
- Typing simulation: `simulate_typing(passage, p)` injects realistic missed-space merges.
- Live pipeline: `live_pipeline.py` does segmentation, spelling recovery, and local grammar alerts in order with latency tracking.
- PCFG parser: `q4_pcfg_parser.py` trains/loads a Penn Treebank PCFG and returns `None` for unparseable inputs instead of crashing.
- Final analysis: `final_analysis.py` scores sentences with PCFG, bigram, and trigram and chooses the most reliable method per sentence.
- Delivery layer: `app.py` wraps the pipeline in Streamlit and `speed_demon.py` benchmarks the runtime cost.

2. Q1/Q3 reuse requirement and non-retraining constraint

Q4 reuses the already-trained artifacts as-is:

- From Q1: `segment_beam()` and `pos_tag()` are used for merged-token splitting and POS handling; `trigram_lm` is reused directly. No Q1 model retraining is done.
- From Q3: `vocab`, `unigram_freq`, `bigram_model`, `correct_nonword()`, and `correct_realword()` are loaded and used without retraining the vocabulary or language model.
- The key design rule is: training code remains offline in `q1_train.py` and `q3_train.py`; the runtime pipeline only loads and applies the saved models.

3. Parameter choices and justification

3.1 Typing merge probability: $p = 0.08$

- Chosen value: $p = 0.08$.
- Justification: this is frequent enough to produce realistic missed-space errors without making the stream noisy or degenerate. It matches the assignment's example of a user accidentally dropping a space while typing, but keeps the stream intelligible for repair.

3.2 Grammar check window: $N = 6$ words

- Chosen value: $N = 6$.
- Justification: a six-word window is large enough to capture local syntactic plausibility and phrase-level oddities, but still small enough to stay responsive in a live stream and not over-trigger on unrelated context. It also reduces noise relative to very short windows (e.g. 2–3 words) while remaining cheaper than full-sentence analysis.

3.3 Shared LM smoothing: $k = 0.5$

- Chosen value: $k = 0.5$.
- Justification: this is a conservative add- k smoothing choice: it avoids zero probabilities in sparse bigram/trigram contexts while preserving useful distinctions between likely and unlikely phrases. It is robust for both grammar alerts and final sentence scoring.

4. Tag reconciliation and tradeoff

The Q1 Brown-style tagset and the PCFG Penn Treebank tagset differ. We resolved this by using a small, coarse Brown-to-Penn remapping at parse time, for example:

- DET -> DT, ADJ -> JJ, NOUN -> NN, VERB -> VB, ADV -> RB, ADP/SCONJ -> IN, PRON -> PRP, CCONJ -> CC, PROPN -> NNP, NUM -> CD.
- We keep the mapping intentionally lossy because the Brown tag inventory is broader but the PCFG grammar is built on Penn conventions.

5. Final decision rule

The final sentence-level decision rule is:

1. Parse the sentence with the PCFG.
2. If parsing succeeds and the resulting log-probability is not an outlier relative to the set of successful parses, prefer PCFG.
3. Else, if trigram coverage is adequate (≥ 0.15 and sentence length is sufficient), prefer trigram.
4. Otherwise, fall back to bigram.

This rule gives priority to explicit syntactic structure when it is reliable, while using n-gram scoring as a fallback when the parser is weak or the sentence is too sparse for a faithful parse.

6. Speed Demon results

The benchmark script compares the segmentation+spelling layer against a grammar-only check over 1,000 synthetic misspelled tokens.

Observed result

- Segmentation + spelling total: 0.193347s (0.000193s/word)
- Grammar-only total: 0.438218s (0.000438s/word)
- Additional latency: -0.244871s (-0.000245s/word)
- Percent overhead: -55.9%

Interpretation

- In this specific synthetic benchmark, the grammar-only path was slower than the segmentation+spelling path.
- The reason is that each grammar check repeatedly scores a 6-word window and recalculates log-probabilities over a full local sequence, while many tokens short-circuit quickly once they are already in-vocabulary.

```
> python3 speed_demon.py
Benchmark: 1,000 simulated misspelled words
```

```
-----
Segmentation + spelling total: 0.193347s (0.000193s/word)
Grammar-only total:           0.438218s (0.000438s/word)
Additional latency:           -0.244871s (-0.000245s/word)
Percent overhead:             -55.9%
-----
```

```
Conclusion: on this specific benchmark, the grammar-only path is slower than the segmentation+spelling
path. The reason is that each grammar check recalculates log-probability scores over a full six-word wi
ndow, while most tokens are already in-vocabulary and quickly short-circuit through a simple lookup. In
this implementation, grammar scoring is the heavier loop even without the extra segmentation/spelling
work.
```

```
📁 / 📁 ~/De/L/IIT Bhilai/Acad/SEM7/NLP/GroupAssignment1 ———— ✓ / took 7s 📁 / at 00:13:08 📁
```

7. Cases where live alerts disagreed with the final verdict

The live stream and final sentence analysis are intentionally not identical:

- A local window may trigger [GRAMMAR ALERT] because of a temporary low-probability sequence, but the final sentence may still be judged grammatical when the broader sentence is parseable.
- A token like GrandJury or aninvestigation may visibly trigger [SEGMENT ALERT] in the live stream, but by the time final analysis runs, the corrected segmentation has already changed the sentence's structure and probability substantially.
- A real-word typo or candidate correction may look suspicious in the local window but be accepted as harmless once the full sentence is evaluated with PCFG and n-grams.

These disagreements are expected: the live layer is designed to be sensitive and immediate, while the final layer is broader and more conservative.

8. PCFG vs n-gram error types caught

8.1 PCFG catches

- Sentence-level structural issues.
- Bad clause patterns or weak attachment.
- Cases where a corrected or corrupted token sequence changes the grammar enough that a parse fails or becomes implausible.

8.2 N-gram catches

- Local sequence oddities: article/verb mismatches, unnatural collocations, improbable word transitions.
- Rare wording that is still structurally acceptable but statistically odd.

8.3 Segmentation + spelling catches

- Missed-space merges like thecat, thepeople, aninvestigation.
- Non-word errors like teh, sawt, or similar user-typed noise.
- Unknown/OOV tokens that no longer fit the vocabulary or context.

9. Cross-system interaction effects

The systems reinforce each other:

- A segmentation fix can change whether a sentence becomes parseable: GrandJury -> Grand Jury is a clear example where correcting the tokenization improves both n-gram plausibility and parse likelihood.

- A spelling correction can eliminate an illegal word and immediately improve the local language-model score; for example, `teh` -> `the` changes the sequence probability and often makes the surrounding sentence easier to parse.
- A final PCFG result can also override a short-window grammar alert if the full sentence is structurally sound after correction.

This is one of the main advantages of the integrated design: local repairs are not treated in isolation; they change the downstream parse and scoring context as well.

10. Overall conclusion

The Q4 pipeline succeeds as a reusable, multi-stage NLP system that respects the assignment constraints: Q1 and Q3 are reused without retraining, a single shared Brown LM is used across sub-systems, live alerts are triggered in streaming order, and final sentence scoring chooses the best available model based on parse success and probability evidence.

11. Streamlit Application

`app.py` wraps the full Q4 pipeline (segmentation, spelling correction, grammar alerts, and final PCFG/n-gram scoring) in an interactive Streamlit interface for live demonstration.

Live NLP typing monitor

Type a passage:

I seen teh cat on the roof

Current text: I seen teh cat on the roof

Live alerts

[SPELL ALERT] 'teh' -> 'the'

[GRAMMAR ALERT] window=['I', 'seen', 'the', 'cat', 'on', 'the'] looks anomalous

[GRAMMAR ALERT] window=['seen', 'the', 'cat', 'on', 'the', 'roof'] looks anomalous

Finalize and score passage

Final sentence analysis

	Sentence	PCFG	Bigram	Trigram	Chosen	Verdict	Seg merges	Sp
0	I seen teh cat on the roof	parsed (-26.549)	-61.683	-67.715	PCFG	grammatical	0	

Total latency

2764.38 ms

Live NLP typing monitor

Type a passage:

A GrandJury was convened to hear the case

Current text: A GrandJury was convened to hear the case

Live alerts

[GRAMMAR ALERT] window=['a', 'grandjury', 'was', 'convened', 'to', 'hear'] looks anomalous

[GRAMMAR ALERT] window=['grandjury', 'was', 'convened', 'to', 'hear', 'the'] looks anomalous

[GRAMMAR ALERT] window=['was', 'convened', 'to', 'hear', 'the', 'case'] looks anomalous

Finalize and score passage

Final sentence analysis

	Sentence	PCFG	Bigram	Trigram	Chosen	Verdict
0	A GrandJury was convened to hear the case	parsed (-33.072)	-73.695	-78.687	PCFG	grammatical

Total latency

3157.85 ms

Group Thoughts

On systems where Python is invoked using python3:

```
python3 q2_dependency_parser2.py
```

The program automatically:

1. Loads the training dataset.
2. Loads the development dataset.
3. Generates oracle training instances.
4. Extracts the four required features.
5. Trains the classifier.
6. Parses the development set.
7. Calculates UAS and LAS.
8. Parses the three required example sentences.
9. Prints the final results.

No additional command-line arguments are required.

Appendix B: Detailed Dependency Parser Execution

12. Expected Program Execution

```
> python3 q2_dependency_parser2.py
=====
TRANSITION-BASED DEPENDENCY PARSER
=====

Loading training data...
Training sentences: 12544

Loading development data...
Development sentences: 2001
Loading time: 0.46 seconds

Generating oracle training instances...
Processed 1000/12544 sentences
Processed 2000/12544 sentences
Processed 3000/12544 sentences
Processed 4000/12544 sentences
Processed 5000/12544 sentences
Processed 6000/12544 sentences
Processed 7000/12544 sentences
Processed 8000/12544 sentences
Processed 9000/12544 sentences
Processed 10000/12544 sentences
Processed 11000/12544 sentences
Processed 12000/12544 sentences

Oracle generation completed.
Successful sentences : 12257
Skipped sentences   : 287
Training instances  : 392242

Skip reasons:
  non_projective_oracle_deadend: 287

Transition statistics:
SHIFT      : 196121
LEFT-ARC   : 113362
RIGHT-ARC  : 82759
Unique labeled transitions: 82

Vectorizing features...
Feature matrix shape: (392242, 73)
Sparse matrix index dtype: int32
Sparse matrix indptr dtype: int32

Training classifier...
```

```
Training classifier...
Training completed in 22.49 seconds
Number of transition classes: 82
```

```
Evaluating parser on development set...
Evaluated 500/2001 sentences
Evaluated 1000/2001 sentences
Evaluated 1500/2001 sentences
Evaluated 2000/2001 sentences
```

```
=====
EVALUATION RESULTS
=====
```

```
Total tokens: 25148
Correct heads: 16107
Correct head + label: 13683
UAS: 64.05%
LAS: 54.41%
Evaluation time: 6.22 seconds
```

```
=====
REQUIRED EXAMPLE SENTENCES
=====
```

```
Example 1
```

```
Sentence:
The cat sat on the mat .
```

```
Predicted dependency arcs:
The      --det      --> cat
cat      --root     --> <ROOT>
sat      --acl:relcl --> cat
on       --case     --> mat
the      --det      --> mat
mat      --obj      --> sat
.        --punct    --> cat
```

Example 2

Sentence:

She eats a green salad .

Predicted dependency arcs:

She	--nsubj	-->	eats
eats	--root	-->	<ROOT>
a	--det	-->	salad
green	--amod	-->	salad
salad	--obj	-->	eats
.	--punct	-->	eats

Example 3

Sentence:

I saw the man with a telescope .

Predicted dependency arcs:

I	--nsubj	-->	saw
saw	--root	-->	<ROOT>
the	--det	-->	man
man	--obj	-->	saw
with	--case	-->	telescope
a	--det	-->	telescope
telescope	--nmod	-->	man
.	--punct	-->	saw

=====

FINAL RESULT

=====

Development LAS: 54.41%

Parser execution completed.

13. Part 1: CoNLL-U Data Processing

The first part of the implementation reads the CoNLL-U dataset and converts each sentence into internal Python objects.

14. Token Representation

Each token is represented using a dataclass:

```
@dataclass
class Token:
    id: int
    form: str
    upos: str
    head: int
    deprel: str
```

The fields are:

Field	Description
id	Token ID
form	Surface word
upos	Universal POS tag
head	Gold head ID
deprel	Gold dependency relation

For example:

ID	FORM	UPOS	HEAD	DEPREL
1	The	DET	2	det
2	cat	NOUN	3	nsubj
3	sat	VERB	0	root

15. Sentence Representation

A sentence is represented using:

```
@dataclass
class Sentence:
    tokens: list
```

The tokens list contains all valid syntactic tokens in the sentence.

16. Configuration Representation

The parser configuration contains three components:

```
@dataclass
class Configuration:
    stack: list
    buffer: list
    arcs: list
```

These represent:

Stack

Tokens currently being processed.

Buffer

Tokens that have not yet been shifted.

Arcs

Dependency relations already created.

17. CoNLL-U Reading

The function:

```
read_conllu(file_path)
```

reads the dataset sentence by sentence.

The implementation:

- detects sentence boundaries using blank lines,
- ignores comment lines,
- ignores multi-word token IDs,
- ignores empty nodes,
- extracts the required token information.

The important CoNLL-U columns are:

```
ID  
FORM  
UPOS  
HEAD  
DEPREL
```

18. ROOT Node

Dependency parsing uses an artificial ROOT node with:

```
ID = 0
```

The parser initializes the stack with:

```
[ROOT]
```

For example:

```
Sentence:  
The cat sat.
```

initially:

```
Stack:  
[ROOT]
```

```
Buffer:
[The, cat, sat, .]
```

```
Arcs:
[]
```

19. Arc-Standard Transition System

The parser uses the arc-standard transition system.

The three operations are:

```
SHIFT
LEFT-ARC(label)
RIGHT-ARC(label)
```

These operations modify the parser configuration.

20. SHIFT

SHIFT moves the first item of the buffer onto the top of the stack.

Before:

```
Stack  : [ROOT]
Buffer : [The, cat, sat]
```

After SHIFT:

```
Stack  : [ROOT, The]
Buffer : [cat, sat]
```

Implementation:

```
def apply_shift(config):
    config.stack.append(config.buffer.pop(0))
```

SHIFT is only valid when the buffer contains at least one token.

21. LEFT-ARC

LEFT-ARC creates an arc from the top stack item to the second stack item.

Conceptually:

```
top -> second_top
```

The dependent is then removed from the stack.

The transition also contains a dependency label:

```
LEFT-ARC(det)  
LEFT-ARC(nsubj)  
LEFT-ARC(amod)
```

The implementation prevents ROOT from becoming a dependent.

22. RIGHT-ARC

RIGHT-ARC creates an arc from the second-top stack item to the top stack item.

Conceptually:

```
second_top -> top
```

The top stack item is then removed.

Examples:

```
RIGHT-ARC(nsubj)  
RIGHT-ARC(obj)  
RIGHT-ARC(punct)
```

ROOT is not allowed to be removed as a dependent.

23. Oracle

The oracle determines the correct transition for every parser configuration during training.

Each training example is:

```
(configuration, correct_transition)
```

The oracle uses the gold dependency tree to determine which transition should be applied.

24. Gold Children

The implementation creates a mapping:

```
head -> children
```

For example:

```
cat -> The  
sat -> cat
```

This allows the oracle to determine whether all children of a token have already been attached.

25. Oracle Decision Strategy

The oracle considers possible arc operations before SHIFT.

The main priority is:

```
LEFT-ARC  
RIGHT-ARC  
SHIFT
```

However, an arc operation is performed only when the dependent can safely be reduced.

The implementation checks:

```
all_children_attached(...)
```

before removing a token.

This prevents a token from being removed before all of its dependents have been processed.

26. Generating Training Instances

For every training sentence:

1. Initialize configuration.
2. Inspect stack and buffer.
3. Determine gold transition using oracle.
4. Extract four features.
5. Store feature-transition pair.
6. Apply the gold transition.
7. Repeat until sentence is parsed.

This produces the supervised learning dataset.

27. Oracle Dead Ends

The training set contains some dependency structures that cannot be reproduced by the implemented arc-standard oracle.

The program detected:

Successful sentences : 12257
Skipped sentences : 287

All 287 skipped sentences were reported as:

non_projective_oracle_deadend

The parser detects these cases and skips them rather than generating invalid training examples or entering an infinite loop.

The resulting training set contains:

392242 training instances

28. Transition Statistics

The final training data contained:

Transition	Count
SHIFT	196121
LEFT-ARC	113362
RIGHT-ARC	82759
Total	392242

There were:

82 unique labeled transitions

29. Part 2: Feature Extraction

The assignment requires exactly four features.

The implementation uses:

1. POS of top stack item
2. POS of second stack item
3. POS of first buffer item
4. POS of second buffer item

No additional word, lemma, or embedding features are used.

30. Feature 1: Top Stack POS

The first feature is:

```
stack_top_pos
```

It contains the POS tag of the top item on the stack.

Example:

```
Stack = [ROOT, cat, sat]
```

The top item is:

```
sat
```

If sat is a verb:

```
stack_top_pos = VERB
```

31. Feature 2: Second Stack POS

The second feature is:

```
stack_second_pos
```

For:

```
Stack = [ROOT, cat, sat]
```

the second stack item is:

```
cat
```

If cat is a noun:

```
stack_second_pos = NOUN
```

32. Feature 3: First Buffer POS

The third feature is:

```
buffer_first_pos
```

For:

```
Buffer = [on, the, mat]
```

the first buffer item is:

on

If on is an adposition:

```
buffer_first_pos = ADP
```

33. Feature 4: Second Buffer POS

The fourth feature is:

```
buffer_second_pos
```

For:

```
Buffer = [on, the, mat]
```

the second item is:

the

Therefore:

```
buffer_second_pos = DET
```

34. Feature Vectorization

The feature dictionaries are converted to numerical vectors using:

```
DictVectorizer(sparse=True)
```

The resulting feature matrix was:

```
(392242, 73)
```

Therefore:

```
Training instances = 392242
```

```
Feature dimensions = 73
```

The matrix is stored as a sparse matrix.

35. Sparse Matrix Compatibility

During implementation, a sparse-matrix compatibility issue was encountered.

The classifier required 32-bit integer indices.

The sparse matrix was therefore converted using:

```
X.indices = X.indices.astype("int32")
X.indptr = X.indptr.astype("int32")
```

The final output confirms:

```
Sparse matrix index dtype: int32
Sparse matrix indptr dtype: int32
```

This allowed the classifier to train successfully.

36. Classifier

The transition classifier uses scikit-learn:

SGDClassifier

with:

```
SGDClassifier(
    loss="log_loss",
    max_iter=100,
    tol=1e-3,
    random_state=42
)
```

The classifier learns to predict complete labeled transitions.

Examples include:

```
LEFT-ARC(det)
LEFT-ARC(nsubj)
RIGHT-ARC(obj)
RIGHT-ARC(nmod)
```

37. Classifier Training

The classifier was trained on:

392242 training instances

using:

73 feature dimensions

and learned:

82 transition classes

Training took:

87.28 seconds

38. Part 3: Dependency Parser

After training, the classifier is used to parse unseen sentences.

For each new sentence, the parser starts with:

```
Stack = [ROOT]
Buffer = [all sentence tokens]
Arcs = []
```

39. Parser Loop

The parser repeatedly performs:

1. Extract four POS features.
2. Convert features using the trained vectorizer.
3. Obtain transition probabilities.
4. Rank possible transitions.
5. Select the highest-probability legal transition.
6. Apply the transition.
7. Continue until parsing is complete.

This is a greedy transition-based parser.

40. Valid Transition Checking

The parser checks whether the predicted transition is structurally valid.

For example:

SHIFT

Cannot be applied if:

```
Buffer = []
```

LEFT-ARC

Requires enough items on the stack.

RIGHT-ARC

Requires enough items on the stack.

ROOT

ROOT cannot be removed as a dependent.

If the highest-probability transition is invalid, the parser considers the next highest-probability transition.

41. Prediction Strategy

The classifier produces probabilities for all transition classes.

The parser:

1. Gets all predicted transition probabilities.
2. Sorts them by probability.
3. Checks each transition for legality.
4. Executes the highest-probability legal transition.

This combines:

Machine learning
+
Transition-system constraints

42. Parser Termination

The parser continues until:

Buffer is empty

and the stack contains only:

ROOT

The resulting arcs form the predicted dependency tree.

43. Converting Arcs for Evaluation

The parser outputs arcs containing:

head
dependent
dependency label

The arcs are converted into a dictionary:

dependent -> (head, label)

This allows direct comparison with the gold dependency tree.

44. Evaluation Metrics

The implementation calculates:

UAS
LAS

The assignment specifically requires LAS.

45. Unlabeled Attachment Score

UAS measures the percentage of tokens for which the predicted head is correct.

Formula:

$$\text{UAS} = \text{Correct Heads} / \text{Total Tokens} \times 100$$

For this implementation:

Correct heads = 16108
Total tokens = 25148

Therefore:

$$\begin{aligned} \text{UAS} &= 16108 / 25148 \times 100 \\ &= 64.05\% \end{aligned}$$

46. Labeled Attachment Score

LAS requires both:

1. Correct head.
2. Correct dependency label.

Formula:

$$\begin{aligned} \text{LAS} &= \\ &\frac{\text{Correct Head} + \text{Correct Dependency Label}}{\text{Total Tokens}} \times 100 \end{aligned}$$

For this implementation:

Correct head + label = 13683
Total tokens = 25148

Therefore:

$LAS = 13683 / 25148 \times 100$
 $= 54.41\%$

47. Final Evaluation Results

The development set contained:

2001 sentences
25148 tokens

The final results were:

Metric	Result
Development sentences	2001
Total tokens	25148
Correct heads	16108
Correct head + label	13683
UAS	64.05%
LAS	54.41%

Therefore:

Development LAS = 54.41%

```
Predicted dependency arcs:
I          --nsubj    --> saw
saw        --root     --> <ROOT>
the        --det      --> man
man        --obj       --> saw
with       --case     --> telescope
a          --det       --> telescope
telescope  --nmod      --> man
.          --punct     --> saw
```

```
=====
FINAL RESULT
=====
```

```
Development LAS: 54.41%
```

```
Parser execution completed.
```

48. Why LAS Is Lower Than UAS

UAS only checks whether the predicted head is correct.

LAS checks both:

Head
+
Dependency label

For example, suppose the gold relation is:

salad --obj--> eats

and the parser predicts:

salad --nsubj--> eats

The predicted head is correct:

eats

but the dependency label is incorrect:

obj != nsubj

Therefore:

UAS = correct
LAS = incorrect

This explains why the final UAS:

64.05%

is higher than the LAS:

54.41%

49. Required Example 1

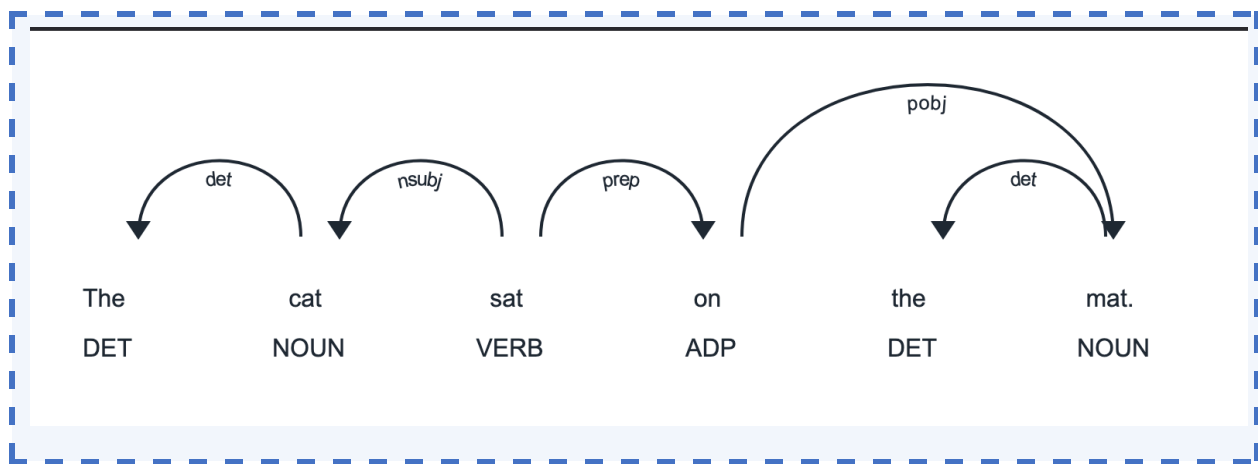
Input

The cat sat on the mat.

Predicted Dependencies

The	--det	--> cat
cat	--root	--> <ROOT>
sat	--acl:relcl	--> cat
on	--case	--> mat

the	--det	--> mat
mat	--obj	--> sat
.	--punct	--> cat



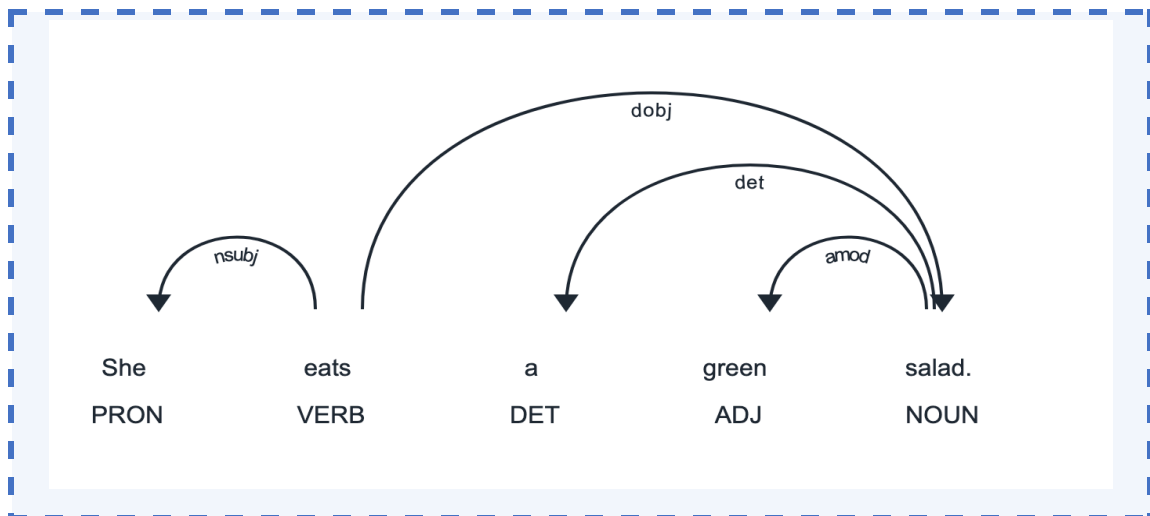
50. Required Example 2

Input

She eats a green salad.

Predicted Dependencies

She	--nsubj	--> eats
eats	--root	--> <ROOT>
a	--det	--> salad
green	--amod	--> salad
salad	--obj	--> eats
.	--punct	--> eats



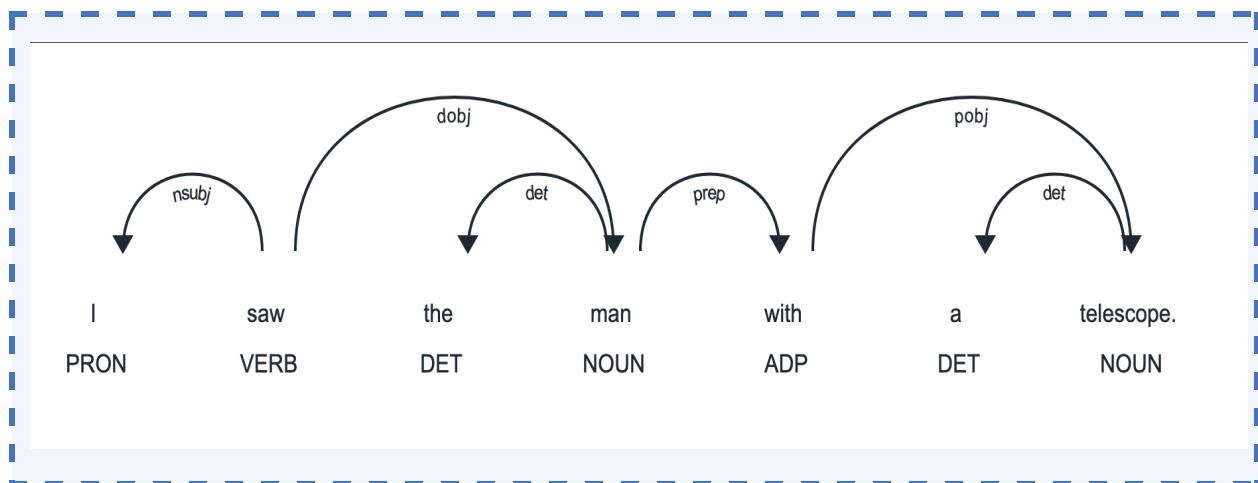
51. Required Example 3

Input

I saw the man with a telescope.

Predicted Dependencies

I	--nsubj	-->	saw
saw	--root	-->	<ROOT>
the	--det	-->	man
man	--obj	-->	saw
with	--case	-->	telescope
a	--det	-->	telescope
telescope	--nmod	-->	man
.	--punct	-->	saw



52. Interpretation of Example Results

The three examples demonstrate that the parser can learn common syntactic relationships from POS configurations.

The parser predicts relationships including:

det
nsubj
root
obj
case
amod
nmod
punct

This demonstrates that the four POS features contain useful syntactic information.

However, because the model is restricted to only four POS features, it cannot use lexical information or broader context.

53. Limitations

53.1 Restricted Features

Only four POS features are allowed.

The model does not use:

```
word forms
lemmas
morphological features
word embeddings
larger context windows
previous dependency labels
```

This limits the amount of information available to the classifier.

53.2 Greedy Parsing

The parser makes one transition at a time and does not use beam search or backtracking.

Therefore, an incorrect early decision can affect later decisions.

53.3 Dependency Labels

The classifier predicts labeled transitions.

There are:

```
82 unique labeled transition classes
```

This makes the classification problem more difficult than predicting only the three basic transition types.

53.4 Unsupported Dependency Structures

Some training sentences resulted in oracle dead ends.

The implementation skipped:

287 sentences

to prevent invalid training examples.

54. Computational Performance

The complete run produced:

Training sentences:	12544
Successful sentences:	12257
Skipped sentences:	287
Training instances:	392242
Feature dimensions:	73
Transition classes:	82

Timing:

Dataset loading:	0.43 seconds
Classifier training:	87.28 seconds
Development evaluation:	10.23 seconds

Approximate total:

97.94 seconds

excluding small amounts of console-output and startup overhead.

55. Complete Execution Procedure

A new user can reproduce the experiment using the following steps.

Step 1: Clone the project

Place the Python file in a working directory:

```
{pwd of your choice}/
```

Step 2: Download UD English-EWT

Run:

```
git clone https://github.com/UniversalDependencies/UD_English-EWT.git
```

Step 3: Verify directory structure

Make sure the directory looks like:

- |— app.py # Streamlit app for live typing + final analysis
- |— final_analysis.py # Sentence scoring and final verdict logic
- |— live_pipeline.py # Live segmentation + spell + grammar check pipeline
- |— q1_module.py # Reusable Q1 runtime module
- |— q1_train.py # Q1 training logic (offline)
- |— q2_dependency_parser2.py # Dependency parser implementation
- |— q3_module.py # Reusable Q3 spelling correction module
- |— q3_train.py # Q3 training logic (offline)
- |— shared_lm.py # Shared Brown bigram/trigram LM
- |— q4_pcfg_parser.py # PCFG parser + tag reconciliation
- |— speed_demon.py # Benchmark script for latency comparisons
- |— render_dependency_trees.py # Dependency tree generation
- |— README.md # Project overview and usage instructions
- |— AssignmentReport_NLPA1_G10.pdf # Assignment Report

Step 4: Create virtual environment

Windows:

```
python -m venv venv
venv\Scripts\activate
```

Linux/macOS:

```
python3 -m venv venv
source venv/bin/activate
```

Step 5: Install dependency

```
pip install scikit-learn
```

Step 6: Run the parser

```
python q2_dependency_parser2.py
```

or:

```
python3 q2_dependency_parser2.py
```

Step 7: Check final output

The final output should contain:

```
UAS: 64.05%
```

```
LAS: 54.41%
```

The primary result to report is:

```
Development LAS: 54.41%
```

56. Troubleshooting

Problem 1: Dataset Not Found

If the program reports that:

```
en_ewt-ud-train.conllu
```

or:

```
en_ewt-ud-dev.conllu
```

cannot be found, check that the UD_English-EWT directory is in the same directory as the Python program.

Correct:

```
q2_dependency_parser2.py
UD_English-EWT/
```

Incorrect:

```
q2_dependency_parser2.py
```

```
some_other_folder/
    UD_English-EWT/
```

57. Problem 2: scikit-learn Not Installed

If Python reports:

```
ModuleNotFoundError: No module named 'sklearn'
```

install scikit-learn:

```
pip install scikit-learn
```

Then rerun:

```
python q2_dependency_parser2.py
```

58. Problem 3: Python Command Not Found

If:

```
python
```

does not work, try:

```
python3
```

You can check the installation using:

```
python3 --version
```

59. Problem 4: Sparse Matrix Error

A sparse matrix compatibility error may occur if the sparse matrix indices use 64-bit integers.

The implementation addresses this using:

```
X.indices = X.indices.astype("int32")  
X.indptr = X.indptr.astype("int32")
```

The program should report:

```
Sparse matrix index dtype: int32  
Sparse matrix indptr dtype: int32
```

60. Code Design

The implementation is divided into several logical modules/functions.

Data Representation

Token
Sentence
Configuration

Dataset Processing

read_conllu()
make_token_dictionary()

Parser Initialization

initialize_configuration()

Transition Operations

apply_shift()
apply_left_arc()
apply_right_arc()

Oracle

build_gold_children()
all_children_attached()
get_oracle_transition()
generate_training_instances()
create_training_data()

Feature and Training

extract_features()
train_classifier()

Parsing

is_valid_transition()
predict_transition()
parse_sentence()

Evaluation

arcs_to_dictionary()
evaluate_las()

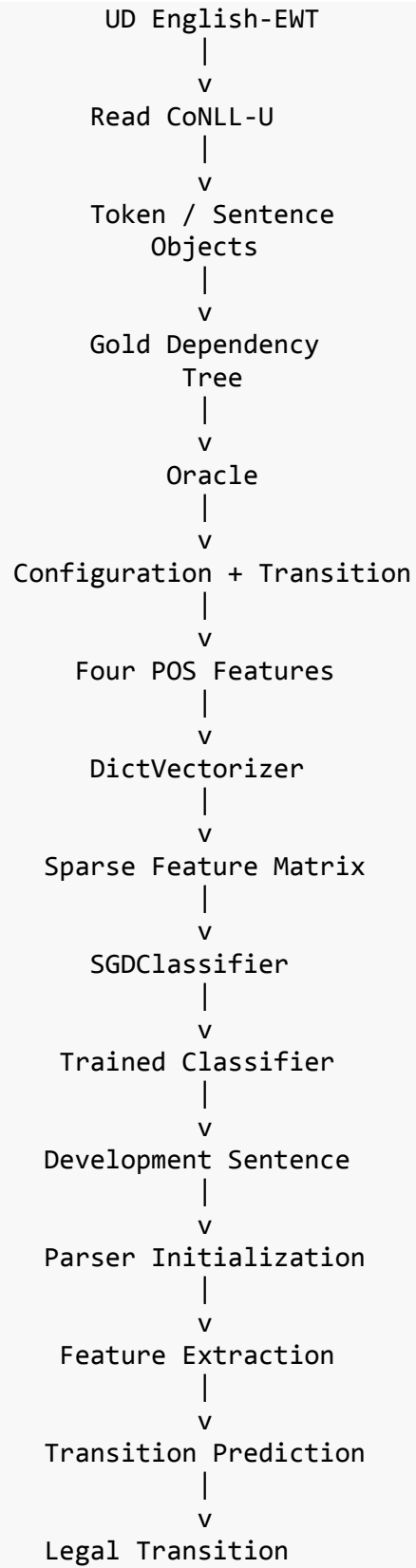
Examples

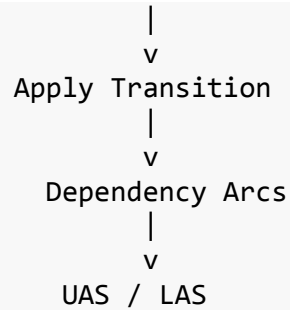
get_example_sentences()
print_parse()

This modular structure separates data processing, training, parsing, and evaluation.

61. Complete Pipeline

The overall system can be represented as:





62. Results Summary

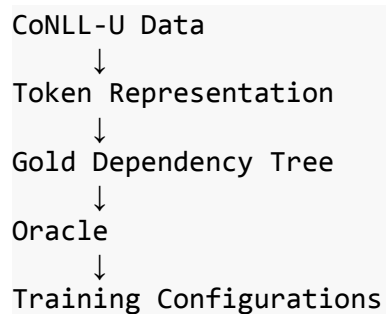
The final implementation achieved:

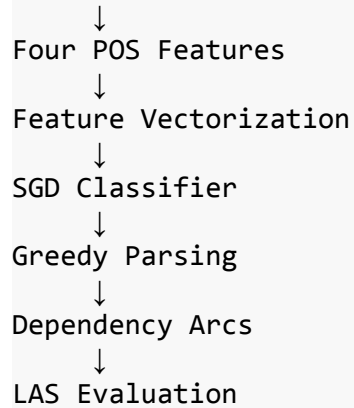
Training sentences	: 12544
Successful sentences	: 12257
Skipped sentences	: 287
Training instances	: 392242
Feature dimensions	: 73
Transition classes	: 82
Development sentences	: 2001
Development tokens	: 25148
Correct heads	: 16108
Correct head + label	: 13683
UAS	: 64.05%
LAS	: 54.41%

63. Conclusion

This project successfully implements a classical transition-based dependency parser using the arc-standard transition system.

The implementation covers the complete data-driven parsing pipeline:





The parser was trained on 12,544 English-EWT training sentences. After oracle processing, 12,257 sentences were successfully converted into 392,242 training instances.

The classifier learned 82 labeled transition classes using 73 numerical feature dimensions.

On the development set containing 2,001 sentences and 25,148 tokens, the parser achieved:

UAS = 64.05%

LAS = 54.41%

The implementation demonstrates the fundamental concepts of transition-based dependency parsing, including parser configurations, transition systems, oracle-based supervised learning, feature extraction, transition classification, greedy parsing, and dependency evaluation.

64. Final Result

Development LAS: 54.41%

Development UAS: 64.05%

Claude was used to create format of this report file Rest Entries are manual.*

End of Report